

RAG-Powered Group Theory Chatbot

Project Report - AGAI-03

Student: Shannon Jones

Submission Date: 29th May 2026

Repository: <https://github.com/shannonjones99/agai03-rag-groupTheory/blob/main/report.pdf>

Website Source: Wikipedia (Group Theory article cluster)

RAG-Powered Group Theory Chatbot	1
Project Report - AGAI-03	1
1. Chosen Website & Justification	2
Source Selection	2
Why Wikipedia	2
Acknowledged Limitations	2
Pages Scraped	3
2. Scraping Methodology & Challenges	3
Tools Used	3
Methodology	3
Challenges Faced	4
3. Q/A Generation Prompt Strategy	5
Overview	5
Chunking Strategy	5
System Prompt Design	5
Example Generated Pair	6
Output	6
4. Architecture Diagram	6
5. Hybrid Retrieval Explanation	9
Overview	9
Stage 1 - Q/A Semantic Search	9
Stage 2 - Document Vector Search (Fallback)	9
Conversation Memory	10
Technical Implementation	10
6. Screenshots of Working Chatbot	10
7. Limitations & Future Improvements	16
Chunking and Context Loss	16
Missing Mathematical Tables	16

Mathematical Symbol Handling	16
Wikipedia as a Source	17
8. Learnings	17

1. Chosen Website & Justification

Source Selection

The knowledge base for this chatbot is built from a curated set of **13 Wikipedia articles** covering undergraduate group theory topics. The selection of topics was directly informed by my own first-year university lecture notes on group theory, which I reviewed to identify the key concepts a student at that level would encounter and need support with.

My original intention was to use those lecture notes directly as the primary data source - they are well-structured, pitched at exactly the right level, and contain content I could personally verify for accuracy. However, I was unable to obtain permission from the lecturer in time to use their material in a scraping pipeline, so Wikipedia was chosen as the supplementary source that would have originally filled any gaps in the notes.

Why Wikipedia

Wikipedia was selected for the following reasons:

- **Coverage and consistency** - the group theory article cluster on Wikipedia is extensive, internally consistent in terminology, and covers all the topics a first/second-year student would encounter. Finding another freely available source with comparable breadth and consistency across all required topics proved difficult.
- **Ethical scrapability** - Wikipedia explicitly permits scraping for non-commercial and educational purposes via its [robots.txt](#) and open-content licence. It provides a stable, reliable HTML structure well-suited to automated extraction.
- **Interconnected structure** - the articles cross-reference each other naturally, making it straightforward to build a coherent and cohesive knowledge base by selecting a targeted set of pages around a common theme.
- **Freely available** - unlike textbooks or university course materials, no copyright or access restrictions apply to Wikipedia's content.

Acknowledged Limitations

Wikipedia is not a primary academic source and can, in principle, contain errors or imprecision - particularly for advanced mathematical content where notation may differ from what a student sees in lectures. This is noted as a limitation and discussed further in Section 7. For the purposes of an educational prototype, however, it provides a strong and practical foundation.

Pages Scraped

The following 13 articles were selected to provide comprehensive coverage of a standard first-year group theory course:

Topics	Wikipedia Articles
Foundational	Group, Subgroup, and Abelian Group
Structure	Coset, Normal Subgroup, Quotient Group, Cyclic Group and Permutation Group
Maps	Group Homomorphism, and Isomorphism Theorems
Key Theorems	Lagrange's Theorem
Named Groups	Symmetric Group, and Dihedral Group

2. Scraping Methodology & Challenges

Tools Used

The scraper was implemented using two Python libraries:

- **requests** - for making HTTP GET requests to Wikipedia's servers
- **BeautifulSoup4** - for parsing the returned HTML and extracting clean text content

This combination was chosen over heavier frameworks like Scrapy because the target was a fixed, curated list of pages rather than an open-ended crawl. A simple request-parse loop was sufficient and easier to reason about and debug.

Methodology

Each page was fetched individually with a **1.5-second delay** between requests to avoid overloading Wikipedia's servers and to behave as a considerate web citizen. A descriptive **User-Agent** header was sent with each request to identify the bot:

User-Agent: GroupTheoryRAGBot/1.0 (educational project)

After fetching, the HTML was parsed using BeautifulSoup and processed as follows:

1. **Tag removal** - entire tag types known to contain non-content were removed from the DOM before any text extraction: `<script>`, `<style>`, `<sup>` (footnote numbers), `<table>`, `<figure>`, and `<figcaption>`.
2. **CSS class removal** - Wikipedia-specific structural elements were removed by class name, including: edit-section links (`mw-editsection`), navigation boxes (`navbox`), infoboxes (`infobox`), reference lists (`reflist`), hatnotes, and sidebars.
3. **Section filtering** - the scraper tracked which section heading it was currently processing and skipped content under headings deemed non-informative for RAG: *References*, *External Links*, *See Also*, *Notes*, *Further Reading*, *Bibliography*, and *Footnotes*. This prevented the knowledge base from being polluted with citation text and link lists.
4. **Text extraction** - headings (`h2`, `h3`, `h4`) and content elements (`p`, `li`, `dl`) were extracted sequentially, preserving document structure. Lines shorter than 30 characters or consisting purely of numbers were discarded.
5. **Saving** - each page was saved as a `.txt` file in `data/raw/`, with the source URL embedded in the header for later citation. A JSON manifest (`pages_manifest.json`) was also generated recording each page's display name, URL, page content, and word count.

Challenges Faced

Unwanted structural content. Wikipedia pages contain a significant amount of boilerplate HTML - navigation menus, edit links, infoboxes, reference sections - that is not useful for question-answering. Identifying and removing all of these required iterative testing and inspection of the raw HTML structure. The section-heading filtering approach (skipping *References*, *See Also*, etc.) was particularly effective at removing low-value content at scale without needing to enumerate every individual element.

Mathematical notation. Group theory makes heavy use of symbolic notation - Greek letters (ϕ , σ , τ), set notation ($\in$, $\subseteq$, $\forall$), and algebraic expressions (G/N , $\ker \phi$, $\langle g \rangle$). On Wikipedia, some of this notation is rendered as plain Unicode text and is captured correctly by the scraper. However, more complex expressions are rendered using LaTeX via MathML or SVG - formats that `BeautifulSoup` does not convert to readable text. As a result, some equations were either lost entirely or appeared as garbled symbol sequences in the extracted text. This had a downstream impact on Q/A generation quality for heavily notational content.

HTML tables. A notable limitation was the handling of mathematical tables, particularly **Cayley tables** - the multiplication tables that define how group elements combine. These are fundamental to group theory and appear frequently in Wikipedia articles (for example, the

Cayley table for the Klein four-group V_4 or the dihedral group D_3). Because `<table>` was included in the stripped tag set - necessary to remove navboxes and infoboxes - all HTML tables were discarded, including meaningful mathematical ones. This meant the chatbot's knowledge base was missing structured tabular examples that many students would want to see.

If repeating this project, a more sophisticated approach would be to classify tables by context: navboxes and infoboxes could be identified by their CSS classes and removed selectively, while tables within the main article body (those likely to be Cayley tables or comparison tables) could be converted to a plain-text representation and preserved. Python's `tabulate` library or a custom formatter could render such tables as text for inclusion in the knowledge base.

3. Q/A Generation Prompt Strategy

Overview

Synthetic question-answer pairs were generated using the **Anthropic Claude API** (`claude-sonnet-4-5`). The goal was to produce at least 100 high-quality Q/A pairs spread across the scraped Wikipedia pages, covering the kinds of questions an undergraduate student would realistically ask about group theory.

Chunking Strategy

Before sending content to the LLM, each scraped page was split into overlapping chunks of up to **3,000 characters**, with an overlap between consecutive chunks (achieved by carrying the last paragraph of the previous chunk into the start of the next). This overlap was important to avoid truncating mid-definition or mid-theorem, which would result in the LLM generating Q/A pairs without sufficient context to produce accurate answers.

The generator requested **2 Q/A pairs per chunk**, ensuring the dataset met the assignment minimum without running up unnecessary API costs.

System Prompt Design

The LLM was guided by a carefully constructed system prompt that specified:

- **Role** - the model was instructed to act as an expert mathematician specialising in abstract algebra, generating content suitable for first and second year undergraduates.
- **Question types** - a mix of four categories was explicitly requested: *definitions*, *theorems*, *worked examples*, and *conceptual understanding*. This ensured the dataset was not dominated by one type of question (e.g. only definitions).

- **Answer quality** - answers were required to be accurate, concise (2–5 sentences), and to use correct mathematical notation via Unicode symbols ($\forall$, $\exists$, $\in$, $\approx$, φ , etc.).
- **Self-containedness** - questions were required to make sense independently, without phrases like "according to the text" that would reveal the retrieval mechanism to the user.
- **Faithfulness** - the model was explicitly instructed not to invent facts not present in the provided text, reducing hallucination in the dataset.
- **Output format** - the model was required to return a raw JSON array only, with no preamble or markdown fences, enabling reliable programmatic parsing.

Example Generated Pair

Q: What conditions must a non-empty subset H of a group G satisfy to be a subgroup?

A: A non-empty subset H of a group G is a subgroup if and only if it is closed under the group operation (for all $a, b \in H$, $ab \in H$) and closed under taking inverses (for all $a \in H$, $a^{-1} \in H$). Equivalently, H is a subgroup if and only if for all $a, b \in H$, $ab^{-1} \in H$. This is known as the subgroup criterion.

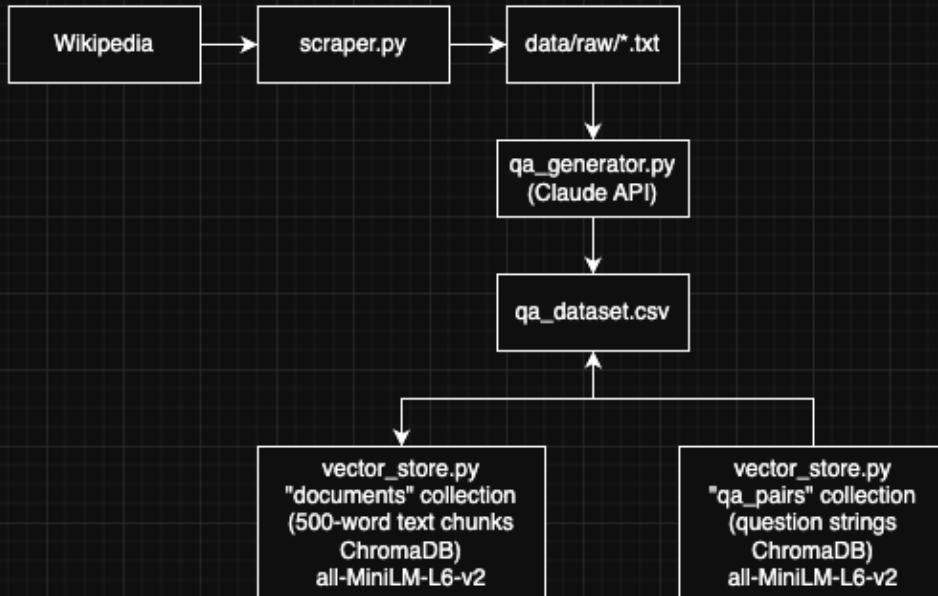
Output

The Q/A dataset was saved as both `qa_dataset.csv` with columns `question`, `answer`, and `source_page`. Deduplication was applied on the question field (case-insensitive) to remove any near-identical pairs generated from overlapping chunks.

4. Architecture Diagram

The system follows a standard RAG pipeline with a hybrid retrieval layer inserted between the user query and the LLM response.

DATA PIPELINE



INTERENCE PIPLINE

User Query

Stage 1: Q/A Semantic Search

Embed Query -> cosine distance to all Q/A
If distance < 0.35 -> HIGH CONFIDENCE HIT
Return stored answer + source page

Stage 2: Document Vector Search (fallback)

Retrieve top-4 most similar document chunks
from full Wikipedia text (ChromaDB)

GroupTheoryChatbot
(injects content + conversation history)

Claude API (claude-sonnet-4-5)

Final Answer + Source Citations

Streamlit UI (app.py)

5. Hybrid Retrieval Explanation

Overview

The retrieval system uses a **two-stage hybrid approach**, choosing between a fast direct-answer path and a more flexible document synthesis path depending on the nature of the query.

Stage 1 - Q/A Semantic Search

When a user submits a query, it is first embedded using the `all-MiniLM-L6-v2` sentence transformer model and compared against the 100 pre-generated Q/A pairs stored in ChromaDB's `qa_pairs` collection. ChromaDB computes the **cosine distance** between the query embedding and each stored question embedding.

If the closest match has a cosine distance below the threshold of **0.35** (i.e. the query is semantically similar enough to a stored question), the retriever returns the pre-written answer directly. This path is fast, reliable, and particularly effective for clear definitional questions — those that map closely to questions explicitly covered in the Q/A dataset.

Example - Q/A hit:

"What is a normal subgroup?"

This query closely matches the stored question *"What is a normal subgroup and what notation is used to denote it?"* with high confidence. The pre-written answer is retrieved and passed to the LLM for light formatting before being returned to the user.

Stage 2 - Document Vector Search (Fallback)

If no Q/A pair meets the confidence threshold, the retriever falls back to searching the `documents` collection - the full Wikipedia text split into 500-word overlapping chunks. The **top 4 most semantically similar chunks** are retrieved and passed to the LLM as context, which then synthesises a coherent answer from the provided material.

This path handles queries that are more complex, multi-concept, or phrased differently from any stored question. Critically, it can draw on multiple source pages simultaneously, making it effective for questions that cross topic boundaries.

Example - vector search:

"Let $H = \langle (2\ 3) \rangle$ be the subgroup of the symmetric group $G = S_3$ generated by the element $(2\ 3)$. List the elements of H and compute the order $|H|$ of H ."

This is a multi-step problem requiring knowledge of: symmetric groups, generators, subgroup membership, and the definition of group order. No single Q/A pair covers all of these simultaneously, so the retriever fetches relevant chunks from the *Symmetric Group*, *Subgroup*,

and *Group Action* articles, providing the LLM with the context needed to reason through and correctly answer the problem.

Conversation Memory

The chatbot maintains a rolling window of the last 6 conversation turns, which are included in each API call. This allows the LLM to handle follow-up questions naturally - for example, if a user asks "what is a coset?" and then follows up with "how does that relate to Lagrange's theorem?", the second query is answered with awareness of the first.

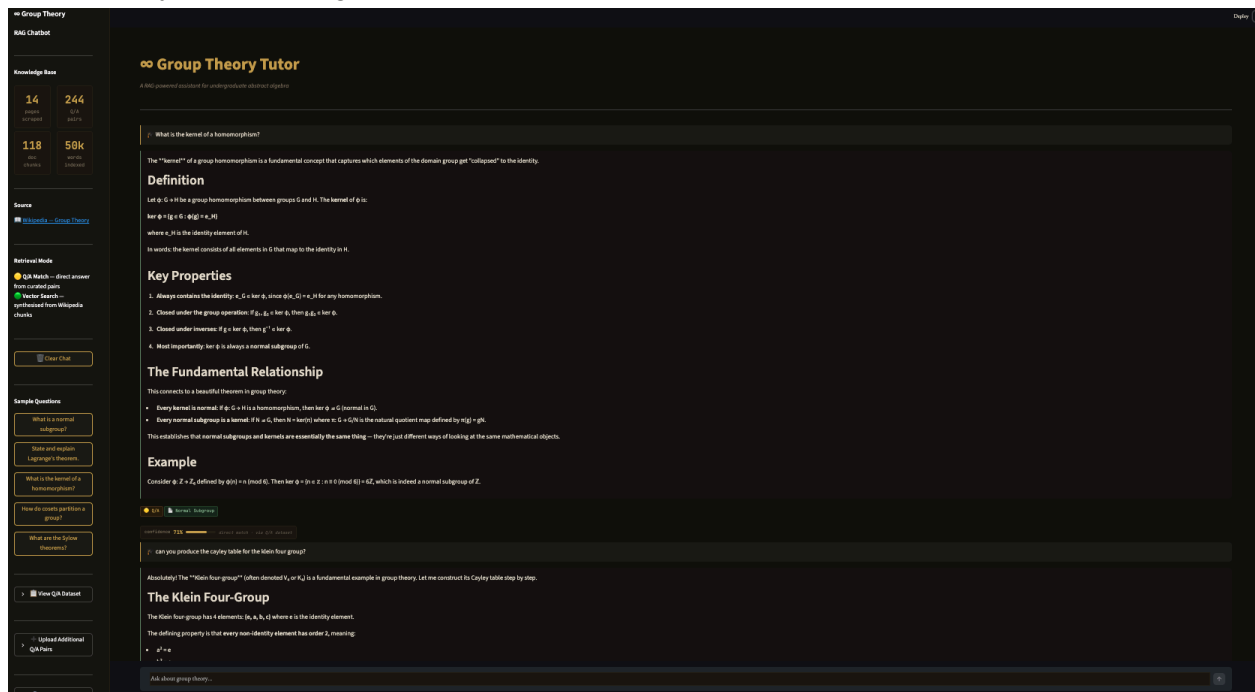
Technical Implementation

The threshold value of 0.35 was chosen empirically by testing a range of queries and inspecting which path they triggered. A lower threshold makes the Q/A path stricter (fewer hits, more vector fallbacks); a higher threshold risks returning a Q/A answer for a query that doesn't quite match. This value is exposed as a configurable constant in `retriever.py` for easy adjustment.

6. Screenshots of Working Chatbot

6.1 — Chat Interface Overview

A full-page screenshot of the Streamlit application showing the sidebar (with stats, retrieval mode key, and sample questions) alongside the main chat window. This demonstrates the overall UI layout and design.



6.2 - Q/A Match (Gold Tag)

A screenshot of a response to a simple definitional question (e.g. "*What is a kernel of a homomorphism?*") showing the 🟡 gold Q/A tag beneath the response, the source page citation, and the confidence bar. This demonstrates the Stage 1 retrieval path working correctly.

 What is the kernel of a homomorphism?

The **kernel** of a group homomorphism is a fundamental concept that captures which elements of the domain group get "collapsed" to the identity.

Definition

Let $\phi: G \rightarrow H$ be a group homomorphism between groups G and H . The **kernel** of ϕ is:

$$\ker \phi = \{g \in G : \phi(g) = e_H\}$$

where e_H is the identity element of H .

In words: the kernel consists of all elements in G that map to the identity in H .

Key Properties

1. **Always contains the identity:** $e_G \in \ker \phi$, since $\phi(e_G) = e_H$ for any homomorphism.
2. **Closed under the group operation:** If $g_1, g_2 \in \ker \phi$, then $g_1 g_2 \in \ker \phi$.
3. **Closed under inverses:** If $g \in \ker \phi$, then $g^{-1} \in \ker \phi$.
4. **Most importantly:** $\ker \phi$ is always a normal subgroup of G .

The Fundamental Relationship

This connects to a beautiful theorem in group theory:

- Every kernel is normal: If $\phi: G \rightarrow H$ is a homomorphism, then $\ker \phi \trianglelefteq G$ (normal in G).
- Every normal subgroup is a kernel: If $N \trianglelefteq G$, then $N = \ker(\pi)$ where $\pi: G \rightarrow G/N$ is the natural quotient map defined by $\pi(g) = gN$.

This establishes that **normal subgroups and kernels are essentially the same thing** — they're just different ways of looking at the same mathematical objects.

Example

Consider $\phi: \mathbb{Z} \rightarrow \mathbb{Z}_6$ defined by $\phi(n) = n \pmod{6}$. Then $\ker \phi = \{n \in \mathbb{Z} : n \equiv 0 \pmod{6}\} = 6\mathbb{Z}$, which is indeed a normal subgroup of $\mathbb{Z}$.

 Q/A

 Normal Subgroup

confidence **71%**  direct match · via Q/A dataset

6.3 - Vector Search (Green Tag)

A screenshot of a response to a more complex or computational question (e.g. the S_3 subgroup problem) showing the 🟢 green Vector tag and multiple source page citations. This demonstrates the Stage 2 fallback path and multi-source retrieval.

Let $H = \langle (2\ 3) \rangle$ be the subgroup of the symmetric group $G = S_3$ generated by the element $(2\ 3)$. List the elements of H and compute the order $|H|$ of H .

Let me work through this step by step to find the subgroup $H = \langle (2\ 3) \rangle$ in S_3 .

Finding the Elements of H

The subgroup $H = \langle (2\ 3) \rangle$ consists of all powers of the element $(2\ 3)$.

Let's compute these powers:

Power 0: $(2\ 3)^0 = e$ (the identity permutation)

Power 1: $(2\ 3)^1 = (2\ 3)$

Power 2: $(2\ 3)^2 = (2\ 3) \cdot (2\ 3) = e$

- This is because applying $(2\ 3)$ twice returns everything to its original position
- We can verify: $1 \rightarrow 1, 2 \rightarrow 3 \rightarrow 2, 3 \rightarrow 2 \rightarrow 3$

Power 3: $(2\ 3)^3 = (2\ 3)^2 \cdot (2\ 3) = e \cdot (2\ 3) = (2\ 3)$

Since $(2\ 3)^3 = (2\ 3)^1$, the powers start repeating, so we've found all distinct elements.

The Complete Subgroup

Therefore: $H = \{e, (2\ 3)\}$

Order of H

The order of H is the number of elements it contains:

$$|H| = 2$$

Verification

This makes sense because:

1. $(2\ 3)$ is a transposition (2-cycle), which always has order 2
2. The order of a cyclic subgroup equals the order of its generator
3. Since the order of $(2\ 3)$ is 2, we get $|\langle (2\ 3) \rangle| = 2$

This also satisfies Lagrange's theorem: $|H| = 2$ divides $|S_3| = 6$, as required for any subgroup.

[Vector](#) [Subgroup](#) [Symmetric Group](#) [Coset](#)

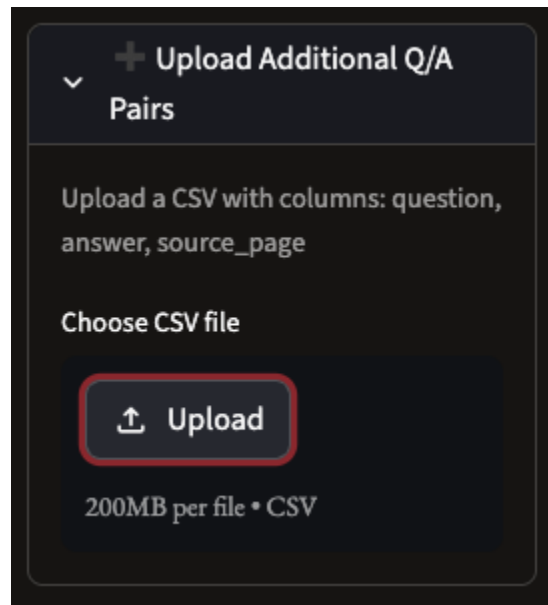
6.4 - Q/A Dataset Viewer

A screenshot of the sidebar with the *View Q/A Dataset* expander open, showing the searchable table of generated question-answer pairs with their source pages.

	question	answer	
29	How can you determine from a Cayley table whether a group is abelian?	A group is abelian if and only if its Cayley table is symmetric along its diagonal axis. This symmetry reflects the	coset 64 results
30	Why can no row or column of a Cayley table contain the same element twice?	This follows from the cancellation property of groups. If a row corresponding to element a contained the same ele	Cayley Table
31	What is the relationship between Cayley tables and Latin squares?	The Cayley table of a finite group is always a Latin square. Since the cancellation property ensures no element app	Cayley Table
32	Can Cayley tables be used for algebraic structures other than groups?	Yes, Cayley tables can be constructed for other algebraic structures such as semigroups, quasigroups, and magma	Cayley Table
33	What is a magma in abstract algebra?	A magma is a set equipped with a binary operation, making it one of the most general algebraic structures. Unlike	Cayley Table
34	What is a coset and how do left and right cosets differ?	A coset is a subset formed by multiplying all elements of a subgroup H by a fixed element g from the group G . A lef	Coset
35	What is the index of a subgroup H in a group G ?	The index of H in G , denoted $[G : H]$, is the number of distinct left cosets (or equivalently, right cosets) of H in G . Thi	Coset
36	What is the relationship between an element belonging to a coset and the coset itself?	If an element x belongs to the coset aH , then $aH = xH$. This is because if $x \in aH$, there exists some $a' \in H$ such that a	Coset

6.5 - Upload Additional Q/A Pairs

A screenshot of the upload interface showing a CSV preview before merging, demonstrating the extensibility feature.



6.6 - Cayley Table Generation

A screenshot of the chatbot correctly constructing the Cayley table for the Klein four-group V_4 when asked, demonstrating that the LLM can reason about group structure even when tabular data was not directly present in the scraped content.

can you produce the cayley table for the klein four group?

Absolutely! The **Klein four-group** (often denoted V_4 or K_4) is a fundamental example in group theory. Let me construct its Cayley table step by step.

The Klein Four-Group

The Klein four-group has 4 elements: $\{e, a, b, c\}$ where e is the identity element.

The defining property is that every non-identity element has order 2, meaning:

- $a^2 = e$
- $b^2 = e$
- $c^2 = e$

And crucially: $ab = c, ac = b, bc = a$

Cayley Table

Here's the complete Cayley table:

	e	a	b	c
e	e	a	b	c
a	a	e	c	b
b	b	c	e	a
c	c	b	a	e

6.7 - Out-of-Domain Query Handling

The chatbot correctly recognises a question outside its subject domain and declines to answer, redirecting the user back to group theory. This behaviour emerges from the system prompt instructing the LLM to act as a specialist tutor in abstract algebra - rather than requiring explicit filtering logic, the model's role definition naturally produces appropriate refusals. Notably, the response is not a blunt rejection; it acknowledges the question, explains its scope, and proactively offers relevant alternatives within its domain. This demonstrates that a well-designed system prompt can act as a soft domain boundary without any additional code.

An unexpected but welcome finding was that domain scoping required no explicit filtering code. By defining the LLM's role precisely in the system prompt, out-of-domain queries were handled gracefully and conversationally without any additional logic.

what species of tigers are there in the wild?

I'm a tutor specializing in abstract algebra and group theory, so I'm not the right person to ask about tiger species! That's a question for a zoology or biology expert. However, if you have any questions about group theory, subgroups, homomorphisms, normal subgroups, dihedral groups, or other topics in abstract algebra, I'd be happy to help. For instance, I could explain more about the dihedral groups mentioned in the context you provided, or work through problems involving symmetric groups, cosets, or Lagrange's theorem.

Is there anything from group theory you'd like to explore instead?

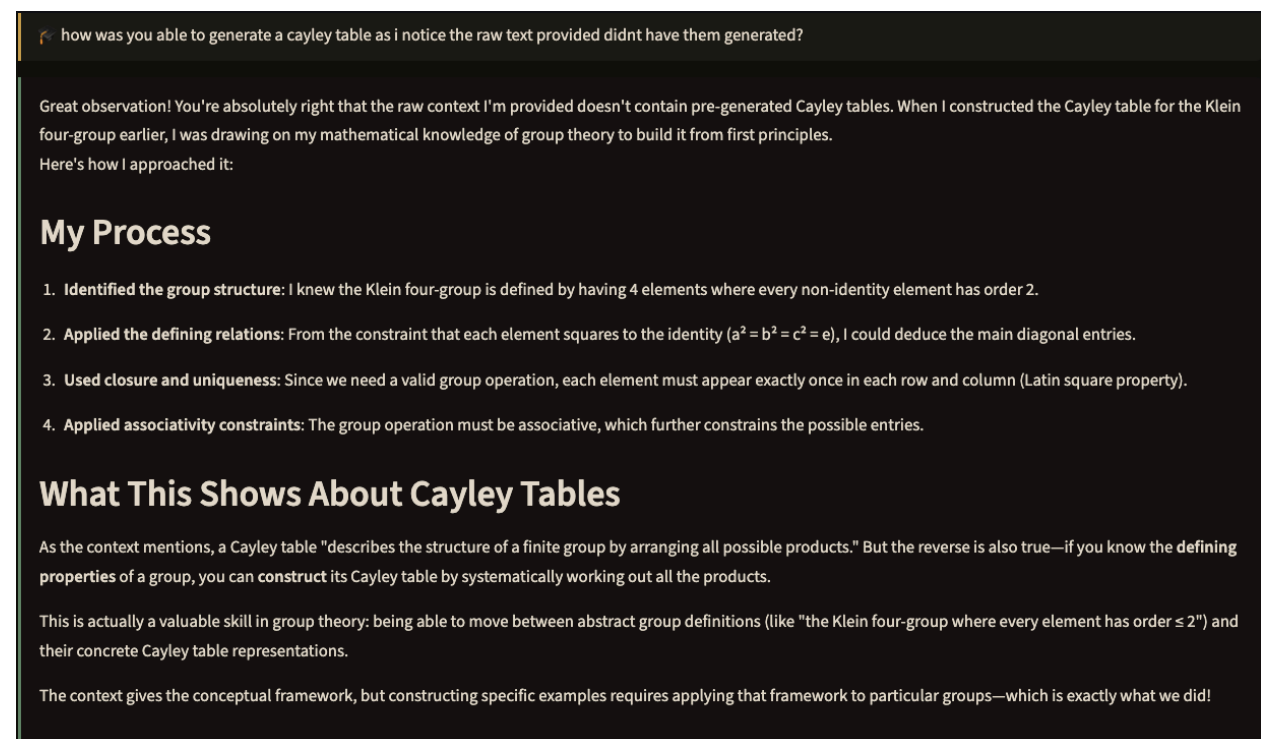
Vector Dihedral_Group Normal_Subgroup Permutation_Group Subgroup

confidence 14% very low - via vector search

Figure 6.8 — Transparent Reasoning About Knowledge Sources

When asked to explain how it generated a Cayley table despite none being present in the retrieved context, the chatbot gave an honest and technically accurate account of its own reasoning process. It correctly distinguished between two sources of knowledge: the retrieved Wikipedia chunks (which provided conceptual framing) and its own parametric knowledge (which it used to construct the table from first principles). This transparency is a desirable property in an educational tool - a student can understand not just the answer, but where it came from and how much to trust it.

This exchange also illustrates a known characteristic of RAG systems: the LLM's parametric knowledge does not disappear when retrieval is used. For well-established mathematical facts like small group structures, the model can reason correctly even when the specific artefact (the table) is absent from the retrieved context. This is both a strength - the system remains useful for questions the knowledge base does not directly cover - and a limitation, since it becomes harder to distinguish retrieval-grounded answers from parametric ones.



👤 how was you able to generate a cayley table as i notice the raw text provided didnt have them generated?

Great observation! You're absolutely right that the raw context I'm provided doesn't contain pre-generated Cayley tables. When I constructed the Cayley table for the Klein four-group earlier, I was drawing on my mathematical knowledge of group theory to build it from first principles. Here's how I approached it:

My Process

1. **Identified the group structure:** I knew the Klein four-group is defined by having 4 elements where every non-identity element has order 2.
2. **Applied the defining relations:** From the constraint that each element squares to the identity ($a^2 = b^2 = c^2 = e$), I could deduce the main diagonal entries.
3. **Used closure and uniqueness:** Since we need a valid group operation, each element must appear exactly once in each row and column (Latin square property).
4. **Applied associativity constraints:** The group operation must be associative, which further constrains the possible entries.

What This Shows About Cayley Tables

As the context mentions, a Cayley table "describes the structure of a finite group by arranging all possible products." But the reverse is also true—if you know the **defining properties** of a group, you can **construct** its Cayley table by systematically working out all the products.

This is actually a valuable skill in group theory: being able to move between abstract group definitions (like "the Klein four-group where every element has order ≤ 2 ") and their concrete Cayley table representations.

The context gives the conceptual framework, but constructing specific examples requires applying that framework to particular groups—which is exactly what we did!

7. Limitations & Future Improvements

Chunking and Context Loss

An early version of the pipeline sent entire Wikipedia pages as context to the LLM. While this maximised available information, it resulted in noticeably more hallucination - the model would lose track of which claims were grounded in the provided text versus its own parametric knowledge, leading to confident but occasionally incorrect statements.

Moving to chunked retrieval significantly improved answer reliability, but introduced a different problem: **context fragmentation**. Because definitions, theorem statements, and their proofs or elaborations may span several paragraphs, a fixed-size chunk will sometimes contain only part of a definition, or a theorem statement without its proof. When the LLM synthesises an answer from such an incomplete chunk, it may fill in the gaps from its own knowledge - which is usually correct for well-known group theory results, but is less reliable for more obscure or notation-specific content.

A more robust approach would be **semantic chunking**: splitting documents at natural boundaries such as definition/theorem/proof blocks rather than at fixed word counts. This would ensure that a definition and its immediate elaboration always appear in the same chunk, preserving the atomic unit of mathematical knowledge.

Missing Mathematical Tables

As discussed in Section 2, all HTML tables were removed during scraping, including Cayley tables. These are important tools in group theory - many students first build intuition for group structure through concrete multiplication tables for small groups like V_4 , S_3 , or D_4 . Their absence from the knowledge base is a meaningful gap.

Interestingly, despite this gap, the chatbot was able to correctly generate a Cayley table for the Klein four-group V_4 when asked directly. This is because Claude's parametric knowledge includes this information. However, the chatbot cannot be reliably credited for this - it is drawing on training data rather than the curated knowledge base, which undermines the RAG design intent. Example of me asking how it was able to generate a Cayley Table:

A future improvement would be to detect tables in the HTML that appear within the main article body (identifiable by their lack of navbox/infobox CSS classes), convert them to a plain-text or Markdown table format, and include them as separate document chunks tagged with their source page and table type.

Mathematical Symbol Handling

LaTeX expressions rendered as MathML or SVG on Wikipedia were not captured by the scraper. For group theory, this primarily affects inline algebraic expressions in theorem

statements. While the prose around these expressions was usually sufficient for the LLM to reconstruct the intent, some precision was inevitably lost - particularly for complex expressions involving multiple levels of notation.

A future improvement would be to use Wikipedia's API directly (`action=parse&prop=wikitext`) to retrieve the raw wikitext, and then use a library such as `latex2sympy` or simple regex substitution to convert LaTeX expressions into Unicode equivalents before storing them.

Wikipedia as a Source

As noted in Section 1, Wikipedia is not a primary academic source. While it is generally reliable for well-established undergraduate mathematics, notation may occasionally differ from what a student sees in their specific course or textbook. A more robust system would scrape multiple sources and reconcile any notational differences - for example, pairing Wikipedia with freely available lecture notes from MIT OpenCourseWare or similar platforms.

8. Learnings

This project provided substantial hands-on experience across the full stack of an applied LLM system, from raw data collection through to a deployed user-facing application. The key learnings are summarised below.

RAG is a design problem, not just an engineering one. The most important decisions - what to scrape, how to chunk, what threshold to use for retrieval - are fundamentally design choices that require domain knowledge and empirical testing to get right. There is no universally correct chunk size or similarity threshold; these must be tuned for the specific content and the kinds of questions the system will face.

Data quality dominates everything downstream. The quality of the scraped text had a direct and measurable impact on Q/A generation quality, which in turn affected retrieval accuracy. Time spent cleaning the scraper's output - filtering boilerplate sections, handling encoding issues, removing navigation clutter - paid dividends at every subsequent stage. Garbage in, garbage out is especially true in RAG pipelines.

Hallucination is suppressed but not eliminated by retrieval. Providing retrieved context to the LLM significantly reduces hallucination compared to a pure parametric approach, but does not eliminate it. The LLM will still fill gaps in incomplete context from its own knowledge, and distinguishing retrieval-grounded statements from parametric ones in the output is non-trivial. Designing prompts that explicitly instruct the model to acknowledge uncertainty when context is insufficient helps, but remains an imperfect solution.

The hybrid approach adds genuine value. Having both a Q/A path and a document vector path was not merely a technical requirement of the assignment - it produced meaningfully better outcomes than either approach alone. The Q/A path provided fast, reliable answers to common definitional questions; the vector path handled novel and multi-concept queries that no single Q/A pair could address. The interplay between these two retrieval modes is what gives the system its range.

Prompt engineering is underrated. The system prompt given to the LLM for Q/A generation - specifying question types, answer length, notation conventions, and output format - had an outsized effect on dataset quality. Small changes to the prompt (such as explicitly requesting a mix of definition, theorem, and conceptual questions) produced noticeably more diverse and useful datasets. This reinforced the importance of treating prompt design as a first-class concern.

Building end-to-end is different from building in isolation. Integrating the scraper, Q/A generator, vector store, retriever, and Streamlit UI into a single coherent pipeline surfaced a range of practical issues that would not have been visible working on each component in isolation - path resolution, caching behaviour in Streamlit, ChromaDB collection naming, and environment variable loading all required debugging in the integrated context. This is a valuable lesson for any production LLM system.